2º FP DESARROLLO DE APLICACIONES MULTIPLATAFORMA (DAM)

Ejercicios Prácticos de Kotlin

Preparación de examen: Bloques 1, 2 y 3 (Fundamentos, Estructuras de Datos y POO)

Esta colección de 6 ejercicios prácticos de programación en consola cubre exclusivamente el temario de los **Bloques 1, 2 y 3** del manual (menús interactivos, control de entradas con `Null Safety`, colecciones como `Map` y `MutableList`, funciones y Programación Orientada a Objetos).

Nivel 1: Condicionales, Bucles y Funciones (Bloques 1 y 2)

Ejercicio 1: Calculadora de tarifas de gimnasio y conversor

> **Conceptos:** `readln()`, `toIntOrNull()`, operador Elvis `?:`, `when` con rangos, bucles `do-while` y funciones con parámetros por defecto.

Enunciado: Crea un programa con un menú interactivo que se repita hasta que el usuario elija la opción de salir:

1. **Calcular cuota mensual:** Solicita la edad del usuario y si desea contratar monitor personal (`S/N`). Crea una función `calcularCuota(edad: Int, conMonitor: Boolean = false): Double` que aplique las siguientes tarifas base usando `when`:
   - Menores de 18 años (`in 0..17`): 20.0 €
   - Adultos (`in 18..64`): 35.0 €
   - Jubilados (`>= 65`): 15.0 €
   - Si `edad` es negativa, devuelve `0.0`. Si contrata monitor, suma 15.0 € al total.
2. **Conversor de unidades de peso:** Solicita un peso en kilogramos (`Double`) y conviértelo a libras (1 kg = 2.20462 libras) y gramos, usando funciones de una sola expresión (`fun kgALibras(kg: Double) = ...`).
3. **Salir.**

Nota: Controla que al leer números por teclado no se rompa el programa utilizando `toIntOrNull() ?: -1` o `toDoubleOrNull() ?: 0.0`.

Ejercicio 2: Validador de credenciales y control de acceso

> **Conceptos:** Tratamiento de `Strings` (`length`, `contains`, `uppercase`, `trim`), `count`, `filter`, bucles `while` y funciones.

Enunciado: Desarrolla un sistema de registro y login para una aplicación empresarial:

1. Crea una función `validarPassword(pass: String): Boolean` que compruebe si una contraseña es segura. Debe devolver `true` solo si cumple **todas** estas condiciones:
   - Tiene al menos 8 caracteres (`length`).
   - No contiene espacios en blanco tras limpiar los extremos (`trim()` , `contains(" ")`).
   - Contiene al menos 2 vocales (usa `pass.lowercase().count { it in "aeiou" } >= 2`).
2. En el `main()` , pide al usuario que cree su nombre de usuario y una contraseña. Si la contraseña no es válida según la función, vuelve a pedirla en un bucle hasta que cumpla los requisitos.
3. Una vez registrado, simula el **inicio de sesión**: el usuario tiene un máximo de **3 intentos** para introducir su usuario y contraseña correctamente. Si acierta, muestra `"Acceso concedido"` y termina el bucle con `break` ; si agota los 3 intentos, muestra `"Cuenta bloqueada"` .

Nivel 2: Estructuras de Datos y Excepciones (Bloque 2)

Ejercicio 3: Gestor de estadísticas de un torneo de eSports

> **Conceptos:** `MutableList` , `Array` bidimensional (matriz), propiedades de listas (`size` , `first()` , `last()`), ordenación (`sortedDescending()`) y bloque `try-catch` .

Enunciado:

1. Crea una matriz bidimensional (`arrayOf` con `intArrayOf`) de 3 filas (jugadores) y 4 columnas (partidas) con puntuaciones predefinidas. Recorre la matriz con dos bucles `for` anidados para mostrar la tabla de puntuaciones por consola y calcula la puntuación total de cada jugador.
2. A continuación, crea una lista mutable `mutableListOf<Int>()` para registrar las puntuaciones de la gran final.
3. Mediante un menú en bucle, permite:
   - **Añadir puntuación:** Lee el número usando `try-catch` capturando `NumberFormatException` . Solo permite puntuaciones entre 0 y 100.
   - **Eliminar puntuación por posición:** Pide el índice y bórralo con `removeAt()` . Captura `IndexOutOfBoundsException` por si el usuario introduce una posición que no existe.
   - **Ver estadísticas:** Muestra todas las puntuaciones ordenadas de mayor a menor (`sortedDescending()`), la puntuación más alta, la más baja y la media total.

Ejercicio 4: Gestión de stock de tienda informática (con Map)

> **Conceptos:** `MutableMap<String, Int>` , recorrido de mapas (`keys` , `values` , `entries`) y ordenación de mapas (`sortedBy` , `sortedByDescending`).

Enunciado: Crea un programa para gestionar el inventario de una tienda donde la **clave** sea el nombre del producto (`String`) y el **valor** sea la cantidad en stock (`Int`):

1. Inicializa un `mutableMapOf` con al menos 4 productos (ej. `"Teclado" to 15`, `"Monitor" to 4`, etc.).
2. Crea un menú con las siguientes opciones:
   - **Añadir o actualizar producto:** Pide el nombre y las unidades. Si el producto ya existe en el mapa (`containsKey` o comprobando si `mapa[nombre] != null`), suma las unidades a las que ya había; si no existe, añádelo.
   - **Vender producto:** Pide el nombre y la cantidad a vender. Si hay stock suficiente, réstalo. Si queda a 0, elimina el producto del mapa.
   - **Mostrar inventario ordenado por stock (de mayor a menor):** Usa `inventario.entries.sortedByDescending { it.value }`.
   - **Mostrar productos ordenados alfabéticamente (A-Z):** Usa `inventario.entries.sortedBy { it.key }`.
   - **Alerta de bajo stock:** Muestra únicamente los productos que tengan menos de 5 unidades.

Nivel 3: Programación Orientada a Objetos (Bloque 3)

Ejercicio 5: Sistema de Gestión de Biblioteca o Videoclub (Herencia y Polimorfismo)

> **Conceptos:** Clases `open`, constructores primarios, bloque `init` con `require()`, `IllegalArgumentException`, herencia, `override` y `super`.

Enunciado:

1. Crea una clase abierta `open class MaterialBiblioteca(val codigo: String, val titulo: String, val precioBaseAlquiler: Double)`:
   - Añade un bloque `init` con `require(precioBaseAlquiler > 0)` que lance un error si el precio es 0 o negativo, y otro `require(titulo.isNotBlank())`.
   - Crea un método `mostrarInfo()` que imprima los datos básicos y un método abierto `open fun calcularPrecioFinal(): Double` que devuelva `precioBaseAlquiler`.
2. Crea dos clases hijas que hereden de `MaterialBiblioteca`:
   - `class Libro(codigo: String, titulo: String, precioBaseAlquiler: Double, val numPaginas: Int)`: Sobrescribe (`override`) `calcularPrecioFinal()`; si el libro tiene más de 500 páginas, suma 1.50 € al resultado de `super.calcularPrecioFinal()`.
   - `class Pelicula(codigo: String, titulo: String, precioBaseAlquiler: Double, val esEstreno: Boolean)`: Sobrescribe `calcularPrecioFinal()`; si es estreno, duplica el precio base.
3. En el `main()`, crea una lista `mutableListOf<MaterialBiblioteca>()`. Intenta añadir varios libros y películas dentro de bloques `try-catch (e: IllegalArgumentException)` (incluye uno con precio negativo para probar que salta el `require`), recorre la lista aplicando **polimorfismo** y muestra el coste total de alquilar todos los materiales de la lista.

Ejercicio 6: Gestión de Pedidos de Restaurante (Data Class y Enum Class)

> **Conceptos:** `enum class`, `valueOf()`, `data class` (`toString()`, `copy()`, `equals()`, desestructuración `componentN()`) y listas de objetos.

Enunciado:

1. Crea un `enum class EstadoReparto { RECIBIDO, COCINANDO, EN_REPARTO, ENTREGADO, CANCELADO }`.
2. Crea una `data class Pedido(val id: Int, val cliente: String, val importe: Double, var estado: EstadoReparto)`.
3. En el `main()`:
   - Crea una lista con 3 pedidos iniciales.
   - **Clonar pedido:** Usa el método `.copy()` sobre el primer pedido para crear un pedido nuevo para el mismo cliente con distinto `id` y estado `RECIBIDO`.
   - **Cambiar estado por teclado:** Pide al usuario el `id` de un pedido y el nuevo estado por texto (`readln().uppercase()`). Usa `EstadoReparto.valueOf(texto)` dentro de un bloque `try-catch (e: IllegalArgumentException)` para actualizar el estado del pedido si es válido.
   - **Resumen desestructurado:** Recorre la lista de pedidos desestructurando cada objeto en el bucle (`for ((id, cliente, importe, estado) in pedidos)`) y muestra por pantalla solo aquellos cuyo estado sea distinto de `CANCELADO`, calculando la facturación total.